

```

[notice] A new release of pip is available: 20.0.1 -> 20.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\WINDOWS\System32> Get-CimInstance Win32_Processor | Select-Object Name,NumberOfCores,NumberOfLogicalProcessors,MaxClockSpeed,L2CacheSize,L3CacheSize | Format-List

Name                : 12th Gen Intel(R) Core(TM) i7-12700H
NumberOfCores       : 14
NumberOfLogicalProcessors : 20
MaxClockSpeed       : 2300
L2CacheSize         : 11776
L3CacheSize         : 24576

PS C:\WINDOWS\System32> python -m cpuinfo
Python Version: 3.14.0.final.0 (64 bit)
Cpuinfo Version: 9.0.0
Vendor ID Raw: GenuineIntel
Hardware Raw:
Brand Raw: 12th Gen Intel(R) Core(TM) i7-12700H
Hz Advertised Friendly: 2.6880 GHz
Hz Actual Friendly: 2.6880 GHz
Hz Advertised: (2688000000, 0)
Hz Actual: (2688000000, 0)
Arch: X86_64
Bits: 64
Count: 20
Arch String Raw: AMD64
L1 Data Cache Size:
L1 Instruction Cache Size:
L2 Cache Size: 65536
L2 Cache Line Size: 1280
L2 Cache Associativity: 7
L3 Cache Size:
Stepping: 3
Model: 154
Family: 6
Processor Type:
Flags: 3dnow, 3dnowprefetch, abm, acpi, adx, aes, apic, avx, avx2, bmi1, bmi2, clflush, clflushopt, clwb, cmov, cx16, cx8, de, dts, erms, est, f16c, fma, fpu, fxsr, gfni, ht, htype,
rsvl, ia64, intel_pt, invpcid, lahf_lm, mca, mce, mmx, monitor, movbe, msr, mtrr, osxsave, pae, pat, pbe, pcid, pclmulqdq, pcdm, pge, pni, popcnt, pse, pse36, rdpid, rdrnd, rdse,
ad, sep, serial, shw, smap, smep, ss, sse, sse2, sse4_1, sse4_2, sse3, tm, tm2, tsc, tscdeadline, umip, vaes, vme, vpclmulqdq, x2apic, xsave, xtptr
PS C:\WINDOWS\System32>

```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\marzh> cd "$env:USERPROFILE\Desktop\lab01"
PS C:\Users\marzh\Desktop\lab01> dir

Karanor: C:\Users\marzh\Desktop\lab01

Mode                LastWriteTime         Length Name
----                -
-a-----         11.09.2026    10:12           2746 task1_audit_commands.txt
-a-----         11.09.2026    10:11           6943 task2_scaling.py
-a-----         11.09.2026    10:11           5086 task3_race.py
-a-----         11.09.2026    10:11           4288 task4_amdahl.py

PS C:\Users\marzh\Desktop\lab01> python task2_scaling.py --calibrate
Calibrating on this machine (single worker)...

LIMIT = 200,000 -> 0.36 s
LIMIT = 500,000 -> 0.74 s
LIMIT = 1,000,000 -> 1.62 s
LIMIT = 2,000,000 -> 4.25 s
LIMIT = 3,500,000 -> 9.14 s

Use LIMIT = 3,500,000 (edit the LIMIT constant at the top).
PS C:\Users\marzh\Desktop\lab01> python task2_scaling.py

TASK 2 - STRONG SCALING BENCHMARK
=====
Host                : Intel64 Family 6 Model 154 Stepping 3, GenuineIntel
Python              : 3.14.0 (Windows)
Logical CPUs seen   : 20
Workload            : count primes below 5,000,000 (trial division)
Runtime             : Python multiprocessing.Pool
Iterations per N    : 3

N=1  run 1: 14.770 s total ( 14.723 s compute)
N=1  run 2: 14.759 s total ( 14.748 s compute)
N=1  run 3: 14.794 s total ( 14.766 s compute)
-> avg 14.774 s | speedup 1.00x | efficiency 100.0%

N=2  run 1: 14.641 s total ( 14.615 s compute)
N=2  run 2: 14.566 s total ( 14.491 s compute)
N=2  run 3: 14.524 s total ( 14.497 s compute)
-> avg 14.557 s | speedup 1.01x | efficiency 50.7%

N=4  run 1: 7.377 s total ( 7.323 s compute)
N=4  run 2: 7.501 s total ( 7.439 s compute)
N=4  run 3: 7.498 s total ( 7.448 s compute)

```

```
Windows PowerShell x Windows PowerShell + v
N=8 run 1: 4.456 s total ( 4.397 s compute)
N=8 run 2: 4.509 s total ( 4.367 s compute)
N=8 run 3: 4.552 s total ( 4.420 s compute)
-> avg 4.506 s | speedup 3.28x | efficiency 41.0%

N=16 run 1: 3.113 s total ( 2.861 s compute)
N=16 run 2: 3.098 s total ( 2.842 s compute)
N=16 run 3: 3.259 s total ( 2.981 s compute)
-> avg 3.157 s | speedup 4.68x | efficiency 29.3%

N=32 run 1: 2.943 s total ( 2.378 s compute)
N=32 run 2: 2.941 s total ( 2.463 s compute)
N=32 run 3: 2.979 s total ( 2.507 s compute)
-> avg 2.954 s | speedup 5.00x | efficiency 15.6%

=====
N Run1 Run2 Run3 AvgTN S_N E_N
=====
1 14.77 14.759 14.794 14.774 1.00x 100.0%
2 14.641 14.506 14.524 14.557 1.01x 50.7%
4 7.377 7.501 7.498 7.459 1.98x 49.5%
8 4.456 4.509 4.552 4.506 3.28x 41.0%
16 3.113 3.098 3.259 3.157 4.68x 29.3%
32 2.943 2.941 2.979 2.954 5.00x 15.6%
=====

Saved -> task2_results.csv
Primes found (identical for every N): 348,513

Carry into TASK 4: T1 = 14.774 s T2 = 14.557 s
Peak speedup: 5.00x at N=32 -> this is your contention wall (Q2.1).
PS C:\Users\marzh\Desktop\lab01> python task2_scaling.py
=====
TASK 2 - STRONG SCALING BENCHMARK
=====
Host : Intel64 Family 6 Model 154 Stepping 3, GenuineIntel
Python : 3.14.0 (Windows)
Logical CPUs seen : 20
Workload : count primes below 5,000,000 (trial division)
Runtime : Python multiprocessing.Pool
Iterations per N : 3

N=1 run 1: 14.373 s total ( 14.320 s compute)
N=1 run 2: 14.382 s total ( 14.351 s compute)
N=1 run 3: 14.297 s total ( 14.192 s compute)
-> avg 14.320 s | speedup 1.00x | efficiency 100.0%

N=2 run 1: 7.258 s total ( 7.232 s compute)
N=2 run 2: 7.319 s total ( 7.286 s compute)
N=2 run 3: 7.386 s total ( 7.357 s compute)
-> avg 7.321 s | speedup 1.96x | efficiency 97.8%

=====
N Run1 Run2 Run3 AvgTN S_N E_N
=====
1 14.373 14.382 14.297 14.32 1.00x 100.0%
2 7.258 7.319 7.386 7.321 1.96x 97.8%
4 4.265 4.427 4.508 4.4 3.25x 81.4%
8 3.096 3.259 3.058 3.138 4.56x 57.1%
16 2.407 2.752 2.658 2.606 5.50x 34.3%
32 2.911 3.108 3.193 3.071 4.66x 14.6%
=====

Saved -> task2_results.csv
Primes found (identical for every N): 348,513

Carry into TASK 4: T1 = 14.32 s T2 = 7.321 s
Peak speedup: 5.50x at N=16 -> this is your contention wall (Q2.1).
PS C:\Users\marzh\Desktop\lab01> python task3_race.py
=====
TASK 3 - UNSYNCHRONIZED SHARED COUNTER
=====
10 workers x 1,000,000 increments
Theoretical expected total: 10,000,000

=====
PART A - 10 THREADS (GIL-serialised - expect NO corruption)
=====
Run Measured Output Error (10^7-Act) Lost % Time ms
=====
#1 10,000,000 0 0.00% 255.8
#2 10,000,000 0 0.00% 291.9
#3 10,000,000 0 0.00% 301.9
#4 10,000,000 0 0.00% 249.3
#5 10,000,000 0 0.00% 330.7
#6 10,000,000 0 0.00% 265.0
#7 10,000,000 0 0.00% 270.9
#8 10,000,000 0 0.00% 350.7
#9 10,000,000 0 0.00% 340.2
#10 10,000,000 0 0.00% 276.0
avg 294.0

=====
```

```
Windows PowerShell x Windows PowerShell + v
-> avg 3.138 s | speedup 4.56x | efficiency 57.1%

N=16 run 1: 2.407 s total ( 2.173 s compute)
N=16 run 2: 2.752 s total ( 2.367 s compute)
N=16 run 3: 2.658 s total ( 2.341 s compute)
-> avg 2.606 s | speedup 5.50x | efficiency 34.3%

N=32 run 1: 2.911 s total ( 2.287 s compute)
N=32 run 2: 3.108 s total ( 2.612 s compute)
N=32 run 3: 3.193 s total ( 2.707 s compute)
-> avg 3.071 s | speedup 4.66x | efficiency 14.6%

=====
N Run1 Run2 Run3 AvgTN S_N E_N
=====
1 14.373 14.382 14.297 14.32 1.00x 100.0%
2 7.258 7.319 7.386 7.321 1.96x 97.8%
4 4.265 4.427 4.508 4.4 3.25x 81.4%
8 3.096 3.259 3.058 3.138 4.56x 57.1%
16 2.407 2.752 2.658 2.606 5.50x 34.3%
32 2.911 3.108 3.193 3.071 4.66x 14.6%
=====

Saved -> task2_results.csv
Primes found (identical for every N): 348,513

Carry into TASK 4: T1 = 14.32 s T2 = 7.321 s
Peak speedup: 5.50x at N=16 -> this is your contention wall (Q2.1).
PS C:\Users\marzh\Desktop\lab01> python task3_race.py
=====
TASK 3 - UNSYNCHRONIZED SHARED COUNTER
=====
10 workers x 1,000,000 increments
Theoretical expected total: 10,000,000

=====
PART A - 10 THREADS (GIL-serialised - expect NO corruption)
=====
Run Measured Output Error (10^7-Act) Lost % Time ms
=====
#1 10,000,000 0 0.00% 255.8
#2 10,000,000 0 0.00% 291.9
#3 10,000,000 0 0.00% 301.9
#4 10,000,000 0 0.00% 249.3
#5 10,000,000 0 0.00% 330.7
#6 10,000,000 0 0.00% 265.0
#7 10,000,000 0 0.00% 270.9
#8 10,000,000 0 0.00% 350.7
#9 10,000,000 0 0.00% 340.2
#10 10,000,000 0 0.00% 276.0
avg 294.0

=====
```

```
Windows PowerShell
PART B - 10 PROCESSES ON SHARED MEMORY, NO LOCK <-- use this table
=====
Run   Measured Output   Error (10*7-Act)   Lost %   Time ms
=====
#1    2,724,565         7,275,435         72.75%   530.7
#2    2,373,824         7,626,176         76.26%   560.4
#3    2,319,640         7,680,360         76.80%   584.6
#4    2,269,705         7,730,295         77.30%   581.0
#5    2,499,955         7,500,045         75.00%   552.5
#6    2,172,572         7,827,428         78.27%   587.3
#7    2,499,613         7,500,387         75.00%   550.7
#8    2,529,669         7,470,331         74.70%   598.5
#9    2,439,439         7,560,561         75.61%   634.6
#10   2,804,950         7,195,050         71.95%   581.5
avg                    576.2
=====
Q3.2 - THE SYNCHRONIZATION PENALTY (same 10 processes, mutex added)
=====
Unlocked = 576.2 ms   result CORRUPTED
Locked   = 52712.5 ms result = 10,000,000 (CORRECT)
Penalty   = 91.49x slower, and now the answer is right.

PS C:\Users\marzh\Desktop\lab01> python task4_amdahl.py 14.320 7.321
=====
BOX 1 - PARALLEL FRACTION DERIVATION
=====
T1 = 14.320 s      T2 = 7.321 s
p = 2 x (T1 - T2) / T1
  = 2 x (14.320 - 7.321) / 14.320
  = 2 x 6.999 / 14.320
  = 0.9775

Sequential fraction (1 - p) = 0.0225 (2.25% of the runtime cannot be parallelised)
=====
BOX 2 - AMDAHL THEORETICAL SPEEDUP CEILING
=====
S_max = 1 / (1 - p) = 1 / 0.0225 = 44.472x
Infinite cores buy you at most 44.47x. Ever.

=====
BOX 3 - AMDAHL 64-CORE PROJECTION (strong scaling)
=====
S_64 = 1 / [(1 - p) + (p / 64)]
      = 1 / [0.0225 + 0.015274]
      = 1 / 0.037769
      = 26.483x
That is 59.6% of the theoretical ceiling, on 64x the hardware.

=====
BOX 4 - GUSTAFSON SCALED SPEEDUP (weak scaling)
=====
S_gustafson = (1 - p) + (p x 64)
             = 0.0225 + 62.5609
             = 62.583x

=====
NUMBERS FOR THE Q4.1 PARAGRAPH
=====
p           = 0.9775
1 - p       = 0.0225
Amdahl S_max = 44.47x
Amdahl S_64  = 26.48x
Gustafson S_64 = 62.58x
Gap between models = 2.4x

The two models disagree because they hold different things
fixed: Amdahl fixes the PROBLEM SIZE and asks how fast 64 cores
finish it, so the serial region becomes a larger and larger share
of the runtime until it is all that is left. Gustafson fixes the
TIME BUDGET and lets the problem grow 64x, so the serial region
stays a constant number of seconds while the parallel work grows
around it. Same algorithm, same p - opposite conclusions.

PS C:\Users\marzh\Desktop\lab01> |
```

```
Windows PowerShell
BOX 2 - AMDAHL THEORETICAL SPEEDUP CEILING
=====
S_max = 1 / (1 - p) = 1 / 0.0225 = 44.472x
Infinite cores buy you at most 44.47x. Ever.

=====
BOX 3 - AMDAHL 64-CORE PROJECTION (strong scaling)
=====
S_64 = 1 / [(1 - p) + (p / 64)]
      = 1 / [0.0225 + 0.015274]
      = 1 / 0.037769
      = 26.483x
That is 59.6% of the theoretical ceiling, on 64x the hardware.

=====
BOX 4 - GUSTAFSON SCALED SPEEDUP (weak scaling)
=====
S_gustafson = (1 - p) + (p x 64)
             = 0.0225 + 62.5609
             = 62.583x

=====
NUMBERS FOR THE Q4.1 PARAGRAPH
=====
p           = 0.9775
1 - p       = 0.0225
Amdahl S_max = 44.47x
Amdahl S_64  = 26.48x
Gustafson S_64 = 62.58x
Gap between models = 2.4x

The two models disagree because they hold different things
fixed: Amdahl fixes the PROBLEM SIZE and asks how fast 64 cores
finish it, so the serial region becomes a larger and larger share
of the runtime until it is all that is left. Gustafson fixes the
TIME BUDGET and lets the problem grow 64x, so the serial region
stays a constant number of seconds while the parallel work grows
around it. Same algorithm, same p - opposite conclusions.

PS C:\Users\marzh\Desktop\lab01> |
```